



**NAMIBIA UNIVERSITY
OF SCIENCE AND TECHNOLOGY**

**DSA521S – DATA STRUCTURES AND ALGORITHMS 1
GROUP MINI-PROJECT 2026**

Mini Project Title

NUST Service Centre Simulation: Designing and Evaluating Data Structures and Algorithms

DUE DATE: 25 September 2026 @ 23:59

Contribution: 20% of the final semester mark

Group size: 5–6 students

The official submission requirements are stated in **Section 12:** Submission Format and Requirements.

1. Project Background

NUST wants to improve the way requests are handled at its student service centre. During a normal day, students arrive for registration assistance, fee enquiries, student card replacement, academic enquiries, document collection, and other administrative services.

Your team has been appointed to design and implement a Campus Service Centre Simulation. The emphasis is not on developing a graphical user interface; it is on **demonstrating correct selection, implementation, and analysis of data structures and algorithms.**

The project uses Arrays, Linked Lists, Queues, Stacks, Searching, Selection Sort, Insertion Sort, Merge Sort and Quick Sort. Most tasks use the service-centre scenario. The stack task is a separate algorithmic exercise and does not need to be integrated into the service-center menu.

2. Programming Language Requirement

The entire project must be implemented using **Java**. All required data structures and algorithms must be implemented by the students as specified in this project. Built-in Java classes or methods that replace the required data structures or sorting algorithms are not permitted.

3. Learning Outcomes

Select appropriate data structures for different computational problems.

Implement and manipulate arrays, linked lists, stacks, and queues.

Design algorithms for insertion, deletion, searching, and traversal.

Implement and trace selected sorting algorithms.

Experimentally compare algorithm performance.

Justify the choice of a data structure or algorithm for a particular problem.

Translate pseudocode into a working program.

Explain and defend an implemented solution.

4. The Problem

Each student arriving at the service centre has the following information:

Student No.	Name	Service Type	Estimated Service Time (min)
221045678	Maria	Registration	12
222034512	Tomas	Student Card	5
223041876	Ndapewa	Fees	8
221067341	Simon	Documents	4

The queue itself represents the order of arrival, so a separate arrival-number (order of arrival) field is not required in the table above. Maria came first, so the arrival number is **1** (**1st**). Your program must use the data structure specified for each task rather than replacing it with a different structure simply because it is easier.

5. PART A — DATA STRUCTURE DESIGN

[30 MARKS]

Task A1 — Waiting Line: Queue [6]

Students must normally be assisted in the order in which they arrive. Implement a Queue to manage students waiting for service.

Required operations:

```
enqueue(student)
dequeue()
peek()
isEmpty()
displayQueue()
```

Demonstrate at least six student arrivals followed by three students being served. Provide a queue diagram, pseudocode, implementation, and a short explanation of why a queue is appropriate for this problem.

Task A2 — Student Service Records: Singly Linked List [8]

Maintain student service records using a singly linked list. Each node must contain a data part storing the student's Student Number, Student Name, Service Type, and Estimated Service Time, together with a next pointer to the next node.

Required operations:

insertStudent()

deleteStudent()

searchStudent()

displayStudents()

Demonstrate insertion at the beginning, insertion at the end, insertion at a specified position (3rd, 4th, etc.), deletion, searching, and traversal.

Draw the linked list before and after at least two operations that change the links between nodes, such as insertion and deletion.

The same Student Service Records linked list and its operations must be used in the integrated system in Part D .

Task A3 — Postfix Expression Evaluation: Stack [6]

Implement a stack-based algorithm to evaluate a postfix arithmetic expression. This is an independent stack exercise and does not form part of the service-centre menu.

Your implementation must use push(), pop() and peek(), and must support the operators +, −, ×, and ÷. When a number is encountered, push it onto the stack. When an operator is encountered, pop the required operands, perform the operation, and push the result back onto the stack.

Example: 5 3 + 2 * evaluates to 16. For one expression, show the stack contents after each major operation and display the final result.

Task A4 — Daily Statistics: Array [5]

Store the service times of students served during the simulated day in an array. Traverse the array algorithmically to determine:

- total students served.
- total service time.
- average service time.
- highest service time.
- lowest service time, and
- number of services longer than 10 minutes.

Built-in shortcuts such as `max()`, `min()`, `sum()` or equivalent functions **may not be used** for these calculations.

Task A5 — Data-Structure Justification [5]

For each of the following structures — Array, Singly Linked List, Stack and Queue — briefly explain why it is appropriate for the task assigned to it. Connect the behaviour of the structure to the actual task; simply defining the structure is not sufficient.

6. PART B — SORTING ALGORITHM CHALLENGE [20 MARKS]

Use the following service-time array for Tasks B1–B4:

[17, 5, 23, 8, 14, 3, 11, 20, 6, 9]

Each algorithm must sort the values into ascending order. **Built-in sorting functions are not allowed.**

Task B1 — Selection Sort [5]

Implement Selection Sort. Count data-value comparisons and swaps. Show the state of the array after the 1st, 2nd, and 3rd passes.

Task B2 — Insertion Sort [5]

Implement Insertion Sort. Count data-value comparisons and shifts. Show the state of the array after the first three passes.

Task B3 — Merge Sort [5]

Implement Merge Sort. Draw the complete divide-and-merge process for the given array and clearly identify the base case.

Task B4 — Quick Sort [5]

Implement Quick Sort. State the pivot-selection rule used and show the pivot, left partition, and right partition for at least the first two partitioning stages.

7. PART C — ALGORITHM EXPERIMENT [15 MARKS]

Experimentally compare Selection Sort, Insertion Sort, Merge Sort and Quick Sort using test arrays of 20, 50, 100 and 500 integers.

Generate the test values programmatically; you are not expected to type hundreds of values manually. For each input size, generate one original array and give each sorting algorithm a copy of exactly the same values so that the comparison is fair.

For this experiment, count only comparisons between data values (not loop-index or boundary checks). Measure only the sorting call: do not include array generation, copying, or printing in the execution time.

Hint for timing in Java: record `System.nanoTime()` immediately before the sorting call and immediately after it.

Execution Time = End Time – Start Time.

Algorithm	Input Size	Number Comparisons	of Execution Time (ns)
Selection Sort	20		
Insertion Sort	20		
Merge Sort	20		
Quick Sort	20		
Selection Sort	50		
Insertion Sort	50		
Merge Sort	50		
Quick Sort	50		
Selection Sort	100		
Insertion Sort	100		
Merge Sort	100		
Quick Sort	100		
Selection Sort	500		

Insertion Sort	500		
Merge Sort	500		
Quick Sort	500		

Additional test: Create an almost-sorted version of the 100-element array by first sorting it in ascending order and then swapping **five pairs of neighbouring values**. Use the same almost-sorted array to test all four sorting algorithms. Compare the number of comparisons and execution time.

Answer the following based on your results and the theory covered in class:

1. Which algorithm performed the fewest comparisons?
2. Which algorithm performed the most comparisons?
3. Which algorithm performed best on the almost-sorted array?
4. Which algorithm appeared to scale best as the input became larger?
5. Did the experimental results generally agree with the theoretical time complexity? Explain.
6. Is execution time alone enough to decide which algorithm is better? Explain.

8. PART D — INTEGRATED SERVICE-CENTRE SYSTEM [10 MARKS]

The Queue, Singly Linked List, Array, and Sorting components of the service-centre scenario must operate within one program. The postfix Stack task in A3 remains a separate exercise.

A suggested menu is:

=====

CAMPUS SERVICE CENTRE

=====

1. Add student to waiting queue
2. Serve next student (remove from queue)
3. Display waiting students
4. Add student service record (Linked List – **insertStudent()**)
5. Display student service records

6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Exit

Select option:

Each option must invoke the appropriate data structure or algorithm. You must be able to explain which structure or algorithm is used by each option and how the corresponding operation works.

Menu Option	Required Structure / Operation
1	Queue — enqueue
2	Queue — dequeue
3	Queue — traversal/display
4	Singly Linked List — insertion
5	Singly Linked List — traversal/display
6	Singly Linked List — search
7	Singly Linked List — deletion
8	Array processing
9	Sorting algorithm(s)
10	Sorting experiment

9. PART E — PSEUDOCODE AND ALGORITHM REPRESENTATION [10 MARKS]

Before or alongside implementation, provide clear pseudocode for the following required operations:

- Queue: enqueue(), dequeue();
- Stack: push(), pop(), postfix evaluation;
- Singly Linked List: insertNode(), deleteNode(), searchNode(), traverseList();
- Sorting: selectionSort(), insertionSort(), mergeSort(), quickSort().

Pseudocode must show the logic of the algorithm and must correspond to the submitted implementation.

10. PART F — PROJECT REPORT [10 MARKS]

Each group must submit **one Project Report in PDF format** documenting the work completed in Parts A–E.

The report must include:

1. **Cover Page** — project title, group number, names and student numbers of all members.
2. **Data Structures** — required diagrams, traces, and brief explanations from Part A.
3. **Sorting Algorithms** — required traces and diagrams from Part B.
4. **Algorithm Experiment** — results tables, almost-sorted test, and answers to the questions in Part C.
5. **Integrated System** — brief explanation of the integrated system and relevant screenshots showing that the program works.
6. **Pseudocode** — required pseudocode from Part E.
7. **GitHub Repository Link**.

The report must be **clear, well organised and consistent with the submitted Java program**. Do not include the complete Java source code in the report.

Mark Allocation: Organisation and presentation (2); completeness of required documentation (3); clarity of explanations, diagrams and results (3); evidence/screenshots of the working system (2). **Total: 10 Marks.**

11. INDIVIDUAL UNDERSTANDING AND ACCOUNTABILITY

Although this is a group project, each group member is responsible for understanding the submitted work. Every member must be able to explain the data structures, algorithms, pseudocode, and source code used in the project.

The lecturer may, where necessary, ask any group member questions about the submitted solution to verify understanding and authorship. This may include explaining an algorithm or section of code, tracing an operation, explaining the choice of a data structure, or answering questions about the group's implementation.

All group members are therefore expected to understand the complete submitted solution and not only the section they personally worked on.

12. SUBMISSION FORMAT AND REQUIREMENTS

12.1 Official eLearning Submission

eLearning/Moodle is the official submission point used for assessment, record-keeping and moderation. A GitHub repository does not replace the required eLearning submission.

- Only **ONE (1)** group representative must submit on behalf of the group on or before **25th September 2026 at 23:59.**
- No hard-copy or email submissions will be accepted.
- Submit one ZIP file named in the format: DSA521S_GroupXX_Project2026.zip.
- The ZIP file must contain: (1) the project report in PDF format; (2) all source-code files required to run the project; and **(3) a README file containing the group number, names and student numbers of all group members, instructions for running the program, and the GitHub repository link. The group member who submits the project on behalf of the group must be clearly identified in the README as: "Submitted by: Student Number – Full Name."**
- The same GitHub repository link must also appear clearly in the project report.
- The eLearning submission must be complete and runnable on its own; files hosted only on GitHub will not be treated as submitted coursework.
- No changes to group membership will be allowed during the final two (2) weeks before the deadline or after submission.
- Groups with fewer than the required number of five **(5)** or six **(6)** may be penalized.
- It is the responsibility of each student to look for group members before the stated timeframe by which membership registration will be closed.

12.2 GitHub Repository and Development History

Each group must maintain one GitHub repository throughout the development period. The repository is used to demonstrate the group's development process and individual participation.

- All group members must contribute commits using identifiable accounts.

- Use meaningful commit messages that describe the work completed, for example: “Added queue enqueue operation”, “Fixed linked-list deletion”, or “Implemented Selection Sort comparison counter”.
- Commits should show progressive development over the project period. A single final-day upload of the complete project is not sufficient evidence of development.
- The repository must contain the source code and any supporting files needed for the project.
- The repository must be accessible to the lecturer for assessment and moderation. If it is private, the group is responsible for granting the required access before the deadline.
- The version submitted on eLearning must correspond to the final version in the repository at the submission deadline.

12.3 Group Identification

The project report and README must clearly state the group number and the full name and student number of every group member. Only work from registered group members will be considered.

13. RESTRICTIONS

Unless specifically authorised by the lecturer, the following are prohibited:

- built-in sorting functions such as `sort()` or equivalent.
- built-in Stack, Queue or Linked-List classes that replace the required student implementation.
- replacing a required data structure with another container simply because it is easier.
- copying or downloading complete implementations and presenting them as the group’s own work
- submitting code that group members cannot explain.

14. AI USE AND ACADEMIC INTEGRITY

Generative AI tools may be used only in accordance with the University’s academic-integrity requirements. Regardless of tools used during development, every group member remains responsible for understanding, explaining, and defending the submitted algorithms and code.

The submitted work must reflect the group’s own development process. The lecturer may use the GitHub history, pseudocode, traces, etc. to verify authorship and understanding.

It is each student’s responsibility to be familiar with and adhere to applicable NUST academic-integrity policies.

15. MARK ALLOCATION

Component	Marks
Part A1 — Queue	6
Part A2 — Singly Linked List	8
Part A3 — Postfix Stack	6
Part A4 — Array Statistics	5
Part A5 — Data-Structure Justification	5
Part B — Four Sorting Algorithms	20
Part C — Algorithm Experiment and Analysis	15
Part D — Integrated Service-Centre System	10
Part E — Pseudocode and Algorithm Representation	10
Part F — Project Report	10
GitHub / Development Process Evidence	5
TOTAL	100

*****END OF PROJECT*****